



Intelligent Business Automation Platform

TECHNICAL MANUAL Theme Customization Setup

Version 5.x

A COMPREHENSIVE PROCESS AUTOMATION SUITE THAT PROVIDES END-TO-END COVERAGE
FOR A VARIETY OF COMPLEX ENTERPRISE BUSINESS PROCESSES

Copyright

Copyright © 2025. All rights reserved.

© All copyright and other property in this document and its contents are confidential and proprietary to the original Organization. No part of these materials should be reproduced, published, transmitted, or distributed in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, or stored in any information storage or retrieval system of any nature nor should the materials be disclosed to third parties without the prior written authorization of the original Organization.

CONFIDENTIAL. DO NOT COPY DARSANA

Table of Contents

1. Contesco Theme Setup.....	4
1.1. Overview	4
1.2. System Requirements	4
1.3. Setting Up Contesco Theme Project	4
1.3.1. Step 1: Update pom.xml Configuration	4
1.3.2. Step 2: Update bnd.bnd File	4
1.3.3. Step 3: Modify Theme Configuration Files.....	4
1.3.4. Step 4: FreeMarker (FTL).....	5
1.3.5. Step 5: Update Login Structure	8
1.3.6. Step 6: Design the Header	8
1.3.7. Step 7: Configure Side Navigation.....	8
1.3.8. Step 8: Modify the Navbar and Scrollbar	8
1.3.9. Step 9: Build and Deploy Theme	8
1.3.10. Step 10: Deploy the WAR File to the Server	9
1.3.11. Step 11: Configure the Deployed Theme.....	9
1.4. Conclusion	9

1. Contesco Theme Setup

1.1. Overview

This document outlines the steps for setting up and deploying the Contesco Theme project. The Platform support team will provide you the source code (Contesco_theme.zip) along with detailed instructions to help you customize and deploy the theme. Comments in the source code files will guide you through the required changes.

1.2. System Requirements

- Maven Version: 3.6.3
- Java Version: 1.8

1.3. Setting Up Contesco Theme Project

The following are the steps to configure and deploy the Contesco theme.

1.3.1. Step 1: Update pom.xml Configuration

1. Open the pom.xml file.
2. Update the configuration with the following values:

```
<groupId>com[REDACTED]groupId>
<artifactId>[REDACTED]
<name>Contesco Theme</name>
```

3. Scroll down to the <build> tag and set the final name of the theme:

```
<finalName>Contesco-Theme</finalName>
```

Note: The "contesco" used here is an example. You can rename it as per your project's needs.

The Contesco source code is available in the Contesco_theme.zip file, which contains all the necessary files and instructions.

1.3.2. Step 2: Update bnd.bnd File

Modify the bnd.bnd file to include the following:

- **Bundle-Name:** Contesco Theme
- **Web-ContextPath:** /contesco-theme
- **Bundle-SymbolicName:** com.platform.portal.theme.contesco

1.3.3. Step 3: Modify Theme Configuration Files

1. Navigate to the directory [REDACTED]
2. Update the **liferay-plugin** [REDACTED]

```
long-description=Pl
module-group-id=com
name=Contesco Theme
```

3. Update the **liferay-look-and-feel.xml** file:

- Modify the **<theme>** tag to the following:

```
<theme id="contesco" name="Contesco Theme">
```

- Configure options to show or hide the header and footer:

```
<settings>
<setting configur
value="true" />
<setting configur
value="true" />
</settings>
```

Note: Additional custom settings can be implemented similarly to header and footer settings.

- Define different color schemes for the theme (optional):

```
<settings>
<color-scheme id="blue" name="blue">
<css-cl
</colo
</sett
```

id - Unique identifier for the theme.
name - Name of the theme.

<css-class> - Defines a CSS class applied to the theme when this scheme is active.

Note: Just like color schemes, we can also configure layouts, permissions, and CSS/JS resources. (Reference: <https://help.liferay.com/hc/en-us/categories/360000868172-Documentation>)

4. Retrieving and updating settings values:

- Include below code in `init_custom.ftl` file

```
<#assign show_header =
getterUtil.getBool
w-header"]!", true)
/> <#assign show_footer =
getterUtil.getBool
w-footer"]!", true)
/>
```

- Including `init_custom.ftl`

```
<#include "${full_path}/init_custom.ftl" />
```

1.3.4. Step 4: FreeMarker (FTL)

1.3.4.1. Objects

ThemeDisplay:

Provided by Platform (as `themeDisplay`), this object holds context information about the current page, layout, and theme.

In the file, it is assigned to a local variable with

```
<#assign theme_display = themeDisplay>.
```

It offers methods to get theme-related paths (like CSS, images, JavaScript), current language, and even user details.

PortletDisplay:

This object (from `portletDisplay`) gives details about the current portlet

- instance—its configuration, title, and state. It is similarly assigned as

```
<#assign portlet_display_name = portlet.getTitle() />
```

making its properties available throughout the template.

Company:

The **company** object represents the company information. It is used to access company-specific details.

Examples in the file include:

```
<#assign company_id = company.getId() />
<#assign company_name = company.getName() />
```

It also provides company logos and

User:

The **user** object holds the current user's data.

Later in the file, properties like the full name, email, greeting, and more are fetched using methods such as

user.getFullName() and **user.getEmailAddress()**.

Conditional checks (like `if (!user_initialized??)`) ensure that user properties are only assigned if they haven't been set already.

1.3.4.2. Variables

FreeMarker uses the **<#assign>** directive to create and initialize variables. This allows you to store reusable data throughout your template.

Example

```
<#assign user = user />
```

1.3.4.3. Conditional

FreeMarker conditional directives include **<#if>**, **<#elseif>**, and **<#else>** directives.

```
<#if layout = "maximized">
<#assign css_class = "maximized" />
</#if>
```

This block checks whether the layout is "maximized" and assigns an extra CSS class.

1.3.4.4. Looping

Although the provided examples use **<#if>** and **<#else>** directives, loops in FreeMarker are implemented using the **<#list>** directive.

```
<#list navigationItems as item>
<li>${item.title}</li>
</#list>
```

This iterates over a collection (e.g., navigation items) and outputs each item.

1.3.4.5. Accessing Web Content in FreeMarker

While the `init.ftl` file mainly sets up the theme's environment, the same principles apply when accessing web content:

Web Content Structures:

When you define a web content structure in Liferay, you set up fields (like title, body, etc.) that become variables in your FreeMarker template.

Example:

```
<h1>${title}</h1>
<div>${body}</div>
```

1.3.4.6. Including External JavaScript Files

Themes typically include external JavaScript files by dynamically generating their URLs:

Generating the JS File URL:

In the `init.ftl`, the JavaScript file is generated by the `theme_display` object as below.

```
<#assign javascript_file_url =
theme_display.getJavaScriptFileURL()
```

This creates a secure, escaped

```
<script src="${javascript_file_url}"
```

This ensures that the script is properly escaped and versioning.

1.3.4.7. Including External CSS Files

Generating the CSS File URL:

The CSS folder path is set as

```
<#assign css_folder = theme_display.getThemeCss() >
```

```
<link rel="stylesheet" href="${css_folder}/${css_main_file}"
/>
```

This method takes care of ensuring the URL is properly escaped for use in the browser.

1.3.4.8. Including External JavaScript Files

1. Create a directory named `javascripts` folder.
2. Include the CSS by `theme_display.getJavaScriptFileURL()`

Example:

```
<link
href="${javascript_file_url}/datatable/datatables.min
.css"
rel="stylesheet"
```

3. Include the JavaScript file by `theme_display.getJavaScriptFileURL()`

Example:

```
<script type="text/javascript"
src="${javascript_file_url}/datatable/datatables.min.js"></script>
```


1.3.5. Step 5: Update Login Structure

1. Open the **main.js** file located in `src/main/webapp/js/`.
2. Modify the login structure as per your requirements.
3. Update the **login.scss** file in `src/main/webapp/css/components/` to customize the login styles.

For example, the company logo (contesco_banner) is added in the **login.scss** .

1.3.6. Step 6: Design the Header

1. Open the **portal_normal.ftl** file located in `src/main/webapp/templates/`.
2. Update the header styles in the **_header.scss** file found in `src/main/webapp/css/`.

For instance, the background color of the dashboard is changed to orange in the **_header.scss** file.

1.3.7. Step 7: Configure Side Navigation

1. Edit the **Navigation.ftl** file located in `src/main/webapp/templates/` to design the side navigation.
2. Update the navigation styles in the **_navigation.scss** file in `src/main/webapp/css/`.

As an example, the side navigation color is changed to match the header (orange) in the **_navigation.scss** file.

1.3.7.1. Including Plugins

1. Create a directory named **lib** (or **library**) inside the **webapps** folder.
2. Include the CSS by adding this code to your HTML:

```
<link
href="${javascript_folder}/dataTables.min
.css"
rel="stylesheet" type="text/css">
```

3. Include the JavaScript by adding:

```
<script type="text/javascript"
src="${javascript_folder}/table/dataTables.min.js"></script>
```

1.3.8. Step 8: Modify the Navbar and Scrollbar

1. Customize the navbar styles by editing the **_navbars.scss** file in `src/main/webapp/css/`.
2. Modify the scrollbar styles in the **_scrollbar.scss** file.
3. Add header logos and icons to the **images** folder located in `src/main/webapp/images/`.

1.3.9. Step 9: Build and Deploy Theme

After completing all design changes, build the WAR file by running the following Maven command from the project directory:

mvn clean install

Once the build completes, the WAR file will be located in the `target/` directory within the project folder:

Path: Sample-project/target/

1.3.10. Step 10: Deploy the WAR File to the Server

1. Deploy the generated WAR file to your server.
2. Once the WAR file is deployed successfully, the Contesco theme will appear in the Platform Control Panel.

1.3.11. Step 11: Configure the Deployed Theme

1. Log in to the Platform.
2. In the Platform title bar, click Administration Settings. Details appear on the left side of the page.
3. Click and expand the Site Menu and then click **Page Tree**.
4. In the Settings (next to the Public Pages Menu), select Look and Feel.
5. Scroll to the bottom and click **Change Current Theme**.
6. Select the newly deployed **Contesco theme** from the list of available themes.

Optional: If you have customized the login page, repeat the steps above to apply the same theme configuration for the login page.

1.4. Conclusion

By following the steps mentioned in this documentation, you will be able to successfully create, customize, and deploy the **Contesco** theme. The project is designed to be flexible, allowing you to modify key elements such as login structure, header, navigation, and overall theme style to match your requirements.